

SoundReal

バックエンド インフラ引き継ぎ書

作成日: 2026 年 4 月 29 日

1. プロジェクト概要

SoundReal は、Wear OS で収集した環境音 (dB・Hz) と位置情報をもとに「音のヒートマップ」と「チェックイン SNS」を提供するアプリです。

このドキュメントはバックエンドサーバー・データベースのインフラ構成と、Railway を使った運用手順をまとめた引き継ぎ書です。

項目	内容
言語	Go 1.22
フレームワーク	gorilla/mux
データベース	PostgreSQL 16
コンテナ	Docker (マルチステージビルド、Alpine Linux)
ホスティング	Railway
API 仕様	OpenAPI 3.0.3 (openapi.yaml)

2. Railway 構成

Railway は GitHub と連携した PaaS (Platform as a Service) です。本プロジェクトでは以下の 2 つのサービスで構成されています。

2-1. サービス構成

サービス名	種別	状態
server	GitHub リポジトリ (Go サーバー)	Online
Postgres	マネージド PostgreSQL 16	Online

2-2. 本番 URL

<https://server-production-5adf.up.railway.app>

この URL が外部 (React Native アプリ・Wear OS) からアクセスするエンドポイントになります。

2-3. 環境変数

server サービスの Variables タブに以下が設定されています。

変数名	値	説明
DATABASE_URL	\${{Postgres.DATABASE_URL}}	Railway が PostgreSQL の接続文字列を自動注入

2-4. 自動デプロイ

GitHub の main ブランチに push すると自動的にビルド&デプロイが走ります。

- コードを修正する
- git add / git commit / git push origin main
- Railway ダッシュボードの Deployments タブで進捗確認

2-5. Railway ダッシュボードへのアクセス

- **https://railway.app:** URL
- **fabulous-flexibility:** プロジェクト名
- **production:** 環境

3. データベース

3-1. テーブル構成

サーバー起動時に db.Init() が自動でテーブルを作成します（手動マイグレーション不要）。

measurements テーブル（環境音の自動収集データ）

カラム名	型	説明
user_id	TEXT	ユーザーID
db	REAL	音量（デシベル）
hz	REAL	周波数（Hz）
latitude	DOUBLE PRECISION	緯度
longitude	DOUBLE PRECISION	経度
created_at	TIMESTAMP	作成日時（自動）

checkins テーブル（ユーザー手動投稿）

カラム名	型	説明
------	---	----

user_id	TEXT	ユーザーID
latitude	DOUBLE PRECISION	緯度
longitude	DOUBLE PRECISION	経度
db	REAL	音量（デシベル）
message	TEXT	投稿メッセージ
created_at	TIMESTAMP	作成日時（自動）

3-2. DB への直接操作

Railway ダッシュボードの Postgres → Database タブの SQL エディタから直接 SQL を実行できます。

テストデータの削除:

```
DELETE FROM measurements WHERE user_id = 'test';
DELETE FROM checkins WHERE user_id = 'test';
```

全件削除（注意: 本番データも消えます）:

```
TRUNCATE measurements;
TRUNCATE checkins;
```

4. API エンドポイント

Method	Path	説明
GET	/health	ヘルスチェック
POST	/measurements	音データ投稿（WearOS から）
GET	/measurements	最新 100 件取得
GET	/measurements/latest	最新 1 件取得
GET	/measurements/area	ヒートマップ用エリア検索
POST	/checkins	Sound Check-In 投稿
GET	/checkins	SNS フィード（最新 50 件）
GET	/areas/{radius_km}/master	エリアマスター取得

詳細なリクエスト／レスポンス仕様は `openapi.yaml` を参照してください。

5. 動作確認コマンド

5-1. macOS / Linux / Git Bash

PowerShell ではなく **Git Bash** を使うと以下のコマンドがそのまま動きます（推奨）。

ヘルスチェック:

```
curl https://server-production-5adf.up.railway.app/health
```

音データ投稿:

```
curl -X POST https://server-production-5adf.up.railway.app/measurements \
-H "Content-Type: application/json" \
-d '{"user_id":"user-001","db":65.5,"hz":440.0,"latitude":35.4437,"longitude":139.6380}'
```

チェックイン投稿:

```
curl -X POST https://server-production-5adf.up.railway.app/checkins \
-H "Content-Type: application/json" \
-d '{"user_id":"user-001","latitude":35.4437,"longitude":139.6380,"db":72.0,"message":"横浜!"}'
```

5-2. Windows (PowerShell)

PowerShell の curl は別物のため **Invoke-RestMethod** を使います。

ヘルスチェック:

```
Invoke-RestMethod -Uri "https://server-production-5adf.up.railway.app/health"
```

音データ投稿:

```
Invoke-RestMethod -Method POST -Uri "https://server-production-5adf.up.railway.app/measurements" `
-ContentType "application/json" `
-Body '{"user_id":"user-001","db":65.5,"hz":440.0,"latitude":35.4437,"longitude":139.6380}'
```

6. Railway トライアル・費用について

現在のプランは **Free Trial** (\$5 クレジット・30 日間)
トライアル開始日: 2026 年 4 月 29 日 → 期限: 2026 年 5 月 29 日
ハッカソン発表日まで余裕で期限内に収まります。

SoundReal 構成 (Go サーバー + PostgreSQL) の推定コスト:

サービス	想定メモリ	12 日間のコスト
Go サーバー	～128MB	約\$0.06
PostgreSQL	～256MB	約\$0.13
合計	～384MB	約\$0.19 (\$5 に対して約 4%)

7. ローカル開発環境

本番ではなくローカルで動作確認したい場合は **Docker Compose** を使います。

起動 (macOS / Linux) :

```
cd server
docker compose up --build
```

起動 (Windows PowerShell) :

```
cd server
docker compose up --build
```

ローカルのエンドポイントは **http://localhost:8080** になります。

8. トラブルシューティング

症状	原因	対処法
FAILED: Error creating build plan with Railpack	Root Directory が未設定	Settings → Root Directory に "server" を設定
DATABASE_URL が接続できない	環境変数の設定ミス	Variables タブで <code>\${Postgres.DATABASE_URL}</code> を確認
PowerShell で curl が動かない	PowerShell の curl はエイリアス	Invoke-RestMethod を使うか Git Bash を使う
デプロイが自動で走らない	main ブランチ以外に push している	git push origin main を確認